

Exercice 04 — TEST-9 a TEST-13 : Finaliser la suite MSPR de la plateforme Code un max

Projet fil rouge : Code un max - Suite de tests automatisés de la plateforme de cours

C'est le **dernier exercice** du projet fil rouge ! Vous allez finaliser et assembler tout le travail réalisé.

Dans cet exercice : Code un max arrive au bout du chantier de fiabilisation de sa plateforme de cours. Les tests unitaires, les E2E en Page Object Model, les donnees Faker, le mocking et le pipeline GitHub Actions sont deja en place (chapitres 1 a 3). Il reste a durcir la suite pour la mise en situation MSPR : la faire tourner plus vite, sur plusieurs navigateurs, produire un rapport lisible et documenter le tout pour que n'importe qui puisse cloner et relancer.

Avant de commencer

L'**Exemple 04** ([exemples/exemple-04/exemple-04.md](#)) t'a fait pratiquer la technique sur un cas neutre. Ici, tu l'appliques au projet **Code un max - Suite de tests automatisés de la plateforme de cours**. Assure-toi de l'avoir lu et suivi avant de commencer.

Vous devez aussi avoir **terminé l'Exercice 03**, car cet exercice s'appuie sur le travail déjà réalisé.

Objectif

Livrer une suite de tests complete integree au pipeline CI/CD, executee en parallele et cross-browser, avec reporting HTML, couverture de code et documentation.

Prérequis

- Python 3.11+ installé
- Git
- VS Code
- Chrome ou Firefox
- Compte GitHub
- Bases de la programmation Python
- Notions de développement web
- Avoir complété les exercices précédents du projet fil rouge

Scénario

L'equipe qualite de Code un max veut pouvoir dire au jury MSPR : voici le depot, voici comment le lancer, voici le rapport de la derniere execution. Tu completes la suite existante sur le projet TEST.

Étapes à réaliser

Étape 1 — Ecrire une nouvelle regle metier en TDD

La plateforme applique une remise de 15 pourcent sur l'inscription a un 3e cours d'un meme apprenant. Cette regle n'est pas encore testee ni codee.

Ce que vous devez faire :

- Ecrire d'abord les tests unitaires de la fonction de calcul du prix d'inscription (cas 1 cours, 2 cours, 3 cours et plus).
- Verifier que la suite est rouge (fonction absente ou incomplete).
- Coder le minimum pour passer au vert, puis refactorer sans casser les tests.

Résultat attendu : Une fonction de tarification couverte par au moins 4 tests unitaires verts, developpee en Red-Green-Refactor.

Piste : Parametrer les tests avec `@pytest.mark.parametrize` pour couvrir les paliers en peu de lignes.

Erreur fréquente : Coder la fonction avant les tests : le jury MSPR attend la preuve du cycle TDD, garde tes commits Red puis Green visibles.

Étape 2 — Rendre la suite E2E cross-browser

Les E2E ne tournent aujourd'hui que sur Chrome. Il faut les faire tourner aussi sur Firefox sans dupliquer le code des tests.

Ce que vous devez faire :

- Parametrer une fixture de driver pour accepter Chrome et Firefox.
- Utiliser la parametrisation pytest pour executer chaque parcours cle sur les deux navigateurs.
- Verifier login, catalogue et inscription sur les deux navigateurs.

Résultat attendu : Chaque scenario E2E s'execute une fois par navigateur, avec un nom de test distinct par navigateur.

Piste : Une fixture parametree (`params=['chrome', 'firefox']`) instancie le bon driver selon la valeur.

Étape 3 — Paralleliser l'execution

La suite complete devient longue. On veut reduire le temps d'execution avec `pytest-xdist`.

Ce que vous devez faire :

- Lancer la suite avec `pytest -n auto` et mesurer le gain de temps.
- Identifier et corriger tout test qui casse uniquement en parallele (etat partage, port, fichier).

Résultat attendu : La suite passe au vert aussi bien en sequentiel qu'en parallele, avec un temps reduit.

Piste : Si un test ecrit dans un fichier ou une DB commune, isole-le avec une fixture par test (`tmp_path`, base dediee).

Erreur fréquente : Conclure que `xdist` est buggy alors que le vrai probleme est un test non isole.

Étape 4 — Traiter les tests instables

Certains tests E2E échouent aléatoirement (chargement lent, animation). Il faut stabiliser sans masquer de vrais bugs.

Ce que vous devez faire :

- Reperer les tests flaky dans les rapports.
- Remplacer les temporisations fixes par des attentes explicites (WebDriverWait).
- N'appliquer pytest-rerunfailures qu'aux tests réellement instables et documenter pourquoi.

Résultat attendu : Les tests flaky sont stabilisés en priorité par des attentes explicites, le rerun n'est qu'un filet de sécurité documenté.

Piste : Marquer un test avec `@pytest.mark.flaky(reruns=2)` plutôt que d'activer le rerun globalement.

Erreur fréquente : Mettre `reruns=3` partout pour cacher un vrai bug : le jury verra l'astuce, corrige la cause d'abord.

Étape 5 — Générer le reporting et la couverture

Produire les livrables attendus pour la MSPR : rapport HTML et mesure de couverture.

Ce que vous devez faire :

- Générer un rapport `pytest-html` auto-contenu.
- Mesurer la couverture du code applicatif avec `pytest-cov` (compatible `xdist`).
- Vérifier que la couverture cible convenue est atteinte.

Résultat attendu : Un `report.html` lisible et un rapport de couverture, tous deux reproductibles en une commande.

Piste : `pytest --cov=app --cov-report=html --html=report.html --self-contained-html -n auto`.

Erreur fréquente : Utiliser `coverage run` avec `-n auto` : les workers ne sont pas comptés, passe par `pytest-cov`.

Étape 6 — Intégrer la suite complète dans GitHub Actions

Le pipeline existe déjà mais ne fait que du lint et des unitaires. Il faut y ajouter la suite complète et publier les artefacts.

Ce que vous devez faire :

- Installer Chrome et Firefox dans le job CI.
- Exécuter la suite complète (unitaires + E2E cross-browser) en parallèle.
- Publier `report.html` et le rapport de couverture comme artefacts téléchargeables.

Résultat attendu : Un push déclenche le pipeline qui exécute toute la suite et rend les rapports téléchargeables depuis l'onglet des exécutions.

Piste : Utiliser `actions/upload-artifact` pour joindre les rapports à chaque run.

Erreur fréquente : Oublier le mode headless : les navigateurs n'ont pas d'affichage dans le runner CI.

Étape 7 — Rédiger le README et livrer

Documenter le projet pour qu'un evaluateur puisse cloner, installer et lancer sans aide.

Ce que vous devez faire :

- Ecrire les sections cloner, installer, lancer les tests, lire le rapport.
- Preciser les prerequis (Python 3.11, navigateurs).
- Joindre le dernier rapport pytest-html au rendu MSPR.

Résultat attendu : Un README clair et un rapport pytest-html a jour, prêts pour la soutenance MSPR.

Piste : Teste ton README sur une machine vierge (ou un container) : si ca ne marche pas au premier essai, corrige-le.

Vérification

Quand vous avez terminé, vérifiez que :

- ☐ La nouvelle regle de tarification a ete developpee en TDD (commits Red puis Green).
- ☐ Les E2E passent sur Chrome et Firefox.
- ☐ La suite passe en sequentiel ET en parallele.
- ☐ Le pipeline GitHub Actions execute tout et publie report.html et la couverture.
- ☐ Le README permet de cloner, installer et lancer sans assistance.

En pratique

Le moment ou les etudiants perdent des points en MSPR, c'est le cross-browser en CI : ca marche en local puis Firefox refuse de demarrer dans le runner faute de binaire ou de mode headless. Fais tourner ton pipeline AVANT la soutenance, pas la veille.

Bonus (Facultatif mais Recommandé)

Ajouter une demo de test de charge Locust sur l'endpoint de login (10 utilisateurs simules) et une passe OWASP ZAP en mode baseline, avec une courte note sur quand ces tests sont pertinents.

Livrable attendu

Depot du projet TEST complet : suite de tests, workflow CI/CD, README et rapport pytest-html de la derniere execution.

Une fois terminé, consultez la **correction** dans [corrections-exercices/correction-exercice-04/correction-exercice-04.md](#) pour comparer votre travail.